A. Abstract

This project focuses on optimizing resource allocation for a retail pet store to maximize profits. By using linear programming, the analysis seeks to determine the optimal mix of products that maximizes the store's profitability. The problem is modeled using Gurobi in Python, incorporating realistic constraints such as budget limits, staff hours, display space, advertising efforts, and product category diversity. Additionally, the analysis includes constraints to ensure balanced advertising efforts across categories and the inclusion of a broad range of products.

B. Data generation in R

My R code generates a synthetic dataset of 1,000 products, each with columns like profit margin, budget required, staff hours, display space, advertising effort, and category. Values are randomly generated, while categories are assigned randomly from "Food," "Toys," or "Accessories." A separate dataset defines resource constraints: a total budget of 50,000, 2,000 staff hours, 500 display units, and 100 advertising units. (code and output, appendix 1)

C. Optimization Problem Setup(Equation written by Latex in R)

<u>Decision Variables</u>

Let: $x_i \geq 0 \quad \text{for } i = 1, 2, \ldots, N$

Where:

xi: Quantity of product i to produce or allocate.,

N: Total number of products.

<u>Objective Function</u>

Maximize the total profit Z: $$Z = \sum_{i=1}^{N} x_i \cdot P_i$$

Where Pi is Profit margin for product i

<u>Constraints (For all the exact number of resource limit, please see appendix 1)</u>

1. Budget Constraint

The total budget required must not exceed the available budget: $\sum_{i=1}^{N} x_i \cdot B_i \leq \text{Budget_Limit}$

Where:

Bi denotes Budget required per unit of product i,

Budget_Limit: Total available budget.

2. Staff Hours Constraint

The total staff hours required must not exceed the available staff hours:

$$\sum_{i=1}^{N} x_i \cdot H_i \leq \text{Staff_Hours_Limit}$$

Where: Hi: Staff hours required per unit of product i.

Staff_Hours_Limit: Total available staff hours.

3. Display Space Constraint

The total display space required must not exceed the available store space:

$$\sum_{i=1}^{N} x_i \cdot S_i \leq \text{Display_Space_Limit}$$

Where:

Si: Display space required per unit of product i.

Display_Space_Limit: Total available display space.

4. Advertising Effort Constraint

The total advertising effort required must not exceed the allowable effort:

$$\sum_{i=1}^{N} x_i \cdot A_i \leq \text{Advertising_Limit}$$

Where:

Ai: Advertising effort required per unit of product i

Advertising_Limit: Total advertising effort allowed.

5. Category Diversity Constraints

For each product category c, at least 20% of the budget must be allocated:

$$\sum_{i \in c} x_i \cdot B_i \geq 0.2 \cdot \text{Budget_Limit}, \quad \forall c$$

Where: c: Categories.

6. Balanced Advertising Constraints

For each product category cc, no category consumes more than 40% of the total advertising effort:

$$\sum_{i \in c} x_i \cdot A_i \leq 0.4 \cdot \text{Advertising_Limit}, \quad \forall c$$

7. Non-Negativity Constraint

$$x_i \geq 0 \quad \forall i$$

D. Solution and analysis with Gurobi in Python (Appendix 2)

Applied 5 methods for the optimization model:

1. Primal Simplex: This method iteratively moves along the edges of the feasible region to improve the objective value. It stops when an optimal vertex is reached, satisfying all constraints.
2. Dual Simplex: This method starts with an infeasible solution and iteratively makes it feasible while improving the dual objective. It is useful when the primal solution is no longer feasible due to changes in constraints.
3. Interior Point: Interior Point methods traverse the interior of the feasible region rather than its edges to find an optimal solution. They are efficient for large-scale problems and use barrier functions to enforce constraints.
4. Automatic Selection: Python will automatically choose the best method based on the problem's structure and size. This ensures computational efficiency and optimal performance without manual intervention.
5. Barrier with Crossover: This method uses logarithmic barrier functions to solve the problem in the interior of the feasible region, achieving high efficiency. Crossover refines the solution to an exact optimal vertex if required for practical applications.

The optimal solution for all 5 methods is the same:

Optimal Profit: 6102.786614213885

Total Budget Used: 30000

Total Staff Hours Used: 512.709032502009

Total Display Space Used: 500

Total Advertising Used: 10

This indicates there is only one single optimal solution for my problem.

E. Conclusion

In conclusion, the objective is to maximize the store's profitability by selecting an optimal allocation of resources for each product while ensuring feasibility and operational efficiency. The constraints reflect practical considerations crucial to store operations, such as ensuring sufficient resources for staffing, budgeting, and advertising efforts, while also meeting strategic goals like maintaining category diversity and fair resource distribution.

This analysis is particularly meaningful for retail operations as it provides a systematic approach to resource allocation, and it ensure the store to make effective decisions. It helps identify the most profitable product mix, and ensures balanced growth across categories, contributing to overall customer satisfaction and business sustainability.

Appendix

1.

```{r}
setwd("C:/Users/Cynthiali0621/Desktop")
```

```{r}
set.seed(999)

# Generate product data
products <- data.frame(
  Product_ID = 1:1000,
  Profit_Margin = runif(1000, 10, 50),  # Profit per unit
  Budget_Required = runif(1000, 50, 200),  # Budget required per unit
  Staff_Hours_Required = runif(1000, 1, 5),  # Staff hours required per unit
  Display_Space_Required = runif(1000, 1, 10),  # Display space required per unit
  Advertising_Effort = runif(1000, 0.5, 3),  # Advertising effort per unit
  Category = sample(c("Food", "Toys", "Accessories"), 1000, replace = TRUE)  # Product category
)

# Generate resource limits
resources <- data.frame(
  Resource = c("Budget", "Staff_Hours", "Display_Space", "Advertising"),
  Limit = c(50000, 2000, 500, 100)  # Total availability
)
```

	Product_ID	Profit_Margin	Budget_Required	Staff_Hours_Required	Display_Space_Required	Advertising_Effort	Category
1	1	25.56286	105.13096	2.917498	3.049563	0.5215455	Food
2	2	33.32243	76.36905	4.666269	8.510880	2.1977840	Accessories
3	3	13.78663	136.96448	4.939025	6.857147	1.3716189	Food
4	4	44.10525	135.26385	4.536770	8.751783	1.1516203	Food
5	5	41.46987	158.99746	1.475800	4.262205	0.7556116	Food
6	6	14.77369	63.74232	4.660305	5.726490	1.8359351	Toys
7	7	34.25788	111.64739	4.310439	8.017941	0.5677460	Toys
8	8	13.23828	179.58020	2.188286	7.066955	2.0204607	Accessories
9	9	25.63090	124.39480	4.261690	1.684791	2.0811853	Toys
10	10	34.77890	170.19355	2.981431	9.717857	2.0076532	Food
11	11	21.42755	113.48684	1.124423	7.571702	0.8138984	Accessories
12	12	32.04491	139.61355	3.442654	3.907989	0.8153278	Accessories
13	13	11.20582	125.54982	4.463755	2.199266	1.6338947	Food
14	14	49.50080	54.76384	4.975904	8.828687	0.5928154	Toys
15	15	14.10460	188.80019	4.628871	1.297614	2.0628202	Food
16	16	41.48852	74.24410	4.266574	4.214953	2.3134235	Toys
17	17	16.66339	135.40098	2.905950	8.962692	1.7121510	Accessories
18	18	42.85024	111.81058	2.550292	4.993168	1.8880761	Toys
19	19	15.24568	152.23492	1.531750	5.488294	1.9931666	Food
20	20	12.79938	101.39489	3.928668	3.817229	2.2854904	Toys
21	21	46.29965	172.41405	4.279751	9.940474	1.4091555	Accessories
22	22	35.35204	54.34050	1.715677	6.635992	1.9565969	Food
23	23	32.13159	148.03247	4.627366	1.367382	1.8891149	Food
24	24	23.68346	109.88592	1.194913	6.310356	1.6034736	Accessories

Showing 1 to 25 of 1,000 entries, 7 total columns

	Resource	Limit
1	Budget	50000
2	Staff_Hours	2000
3	Display_Space	500
4	Advertising	100

2.

```python
from gurobipy import Model, GRB, quicksum
import pandas as pd

file_path = r"D:\240\homework\products.csv"
products = pd.read_csv(file_path)
file_path1 = r"D:\240\homework\resources.csv"
resources = pd.read_csv(file_path1)

resource_limits = dict(zip(resources['Resource'], resources['Limit']))

# Maximum advertising limits by category
max_advertising = {
    "Food": 0.4 * resource_limits['Advertising'],
    "Toys": 0.4 * resource_limits['Advertising'],
    "Accessories": 0.4 * resource_limits['Advertising']
}

# Define a function to solve the model using different methods
def solve_model(method_name, method_value):  1 usage
    """
    Solves the model using a specified method.
    """
    # Initialize the model
    model = Model(f"PetStore_{method_name}")
    model.setParam( paramname: 'Method', method_value)  # Set solver method

    # Decision Variables: Quantity of each product to produce
    x = model.addVars(products['Product_ID'], lb=0, vtype=GRB.CONTINUOUS, name="x")
```

```python
    # Objective Function: Maximize profit
    model.setObjective(
        quicksum(x[i] * products.loc[products['Product_ID'] == i, 'Profit_Margin'].values[0] for i in products['Product_ID']),
        GRB.MAXIMIZE
    )

    # Constraints
    # 1. Budget Constraint
    model.addConstr(
        quicksum(x[i] * products.loc[products['Product_ID'] == i, 'Budget_Required'].values[0] for i in products['Product_ID']) <= resou
        name: "Budget"
    )

    # 2. Staff Hours Constraint
    model.addConstr(
        quicksum(x[i] * products.loc[products['Product_ID'] == i, 'Staff_Hours_Required'].values[0] for i in products['Product_ID']) <=
        name: "Staff_Hours"
    )

    # 3. Display Space Constraint
    model.addConstr(
        quicksum(x[i] * products.loc[products['Product_ID'] == i, 'Display_Space_Required'].values[0] for i in products['Product_ID']) <
        name: "Display_Space"
    )
```

```python
    # 4. Advertising Constraint
    model.addConstr(
        quicksum(x[i] * products.loc[products['Product_ID'] == i, 'Advertising_Effort'].values[0] for i in products['Product_ID']) <= re
        name: "Advertising"
    )

    # 5. Category Diversity Constraints (at least 20% of budget for each category)
    categories = products['Category'].unique()
    for category in categories:
        category_products = products[products['Category'] == category]
        model.addConstr(
            quicksum(x[i] * products.loc[products['Product_ID'] == i, 'Budget_Required'].values[0] for i in category_products['Product_
            name: f"CategoryDiversity_{category}"
        )

    # 6. Balanced Advertising Constraints (no category consumes more than 40% of advertising effort)
    for category in categories:
        category_products = products[products['Category'] == category]
        model.addConstr(
            quicksum(x[i] * products.loc[products['Product_ID'] == i, 'Advertising_Effort'].values[0] for i in category_products['Produc
            name: f"Advertising_{category}"
        )
```

```python
    # Solve the model
    model.optimize()

    # Output results
    if model.status == GRB.OPTIMAL:
        variables = model.getVars()
        num_selected_products = len([var for var in variables if var.x > 0])
        total_budget_used = sum(var.x * products.loc[products['Product_ID'] == int(var.varName.split('[')[1].split(']')[0]), 'Budget_Re
        total_staff_hours_used = sum(var.x * products.loc[products['Product_ID'] == int(var.varName.split('[')[1].split(']')[0]), 'Staf
        total_display_space_used = sum(var.x * products.loc[products['Product_ID'] == int(var.varName.split('[')[1].split(']')[0]), 'Di
        total_advertising_used = sum(var.x * products.loc[products['Product_ID'] == int(var.varName.split('[')[1].split(']')[0]), 'Adve

        print(f"Method: {method_name}")
        print(f"Optimal Profit: {model.objVal}")
        print(f"Number of Selected Products: {num_selected_products}")
        print(f"Total Budget Used: {total_budget_used}")
        print(f"Total Staff Hours Used: {total_staff_hours_used}")
        print(f"Total Display Space Used: {total_display_space_used}")
        print(f"Total Advertising Used: {total_advertising_used}")

        allocation = {var.varName: var.x for var in model.getVars() if var.x > 0}
        print("Sample Allocation (Top 10):", dict(list(allocation.items())[:10]))
        print("-" * 50)
    else:
        print(f"Method: {method_name} resulted in infeasibility or no solution.")
        print("-" * 50)
```

```python
# Solve the model using different methods
methods = {
    'Primal Simplex': 0,
    'Dual Simplex': 1,
    'Interior Point': 2,
    'Automatic Selection': -1,
    'Barrier with Crossover': 3
}

for method_name, method_value in methods.items():
    solve_model(method_name, method_value)
```

Partial Output:

```
Concurrent LP optimizer: primal simplex, dual simplex, and barrier
Showing barrier log only...

Ordering time: 0.00s

Barrier statistics:
 AA' NZ     : 3.300e+01
 Factor NZ  : 5.500e+01
 Factor Ops : 3.850e+02 (less than 1 second per iteration)
 Threads    : 1

Barrier performed 0 iterations in 0.00 seconds (0.00 work units)
Barrier solve interrupted - model solved by another algorithm


Solved with primal simplex
Iteration    Objective       Primal Inf.    Dual Inf.      Time
      31    6.1027866e+03   0.000000e+00   0.000000e+00      0s

Solved in 31 iterations and 0.00 seconds (0.00 work units)
Optimal objective  6.102786614e+03
Method: Barrier with Crossover
Optimal Profit: 6102.786614213883
Number of Selected Products: 5
Total Budget Used: 30000.0
Total Staff Hours Used: 512.709032502009
Total Display Space Used: 500.0
Total Advertising Used: 100.0
Sample Allocation (Top 10): {'x[84]': 39.46390443137948, 'x[299]': 24.264560357947243, 'x[330]': 52.035718786673385, 'x[763]': 5.994686477782215, 'x[854]': 51.52020040255822}
```

```
Set parameter Method to value 1
Gurobi Optimizer version 11.0.3 build v11.0.3rc0 (win64 - Windows 10.0 (19045.2))

CPU model: Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz, instruction set [SSE2|AVX|AVX2]
Thread count: 6 physical cores, 12 logical processors, using up to 12 threads

Optimize a model with 10 rows, 1000 columns and 6000 nonzeros
Model fingerprint: 0x789417cf
Coefficient statistics:
  Matrix range     [5e-01, 2e+02]
  Objective range  [1e+01, 5e+01]
  Bounds range     [0e+00, 0e+00]
  RHS range        [4e+01, 5e+04]
Presolve time: 0.00s
Presolved: 10 rows, 1000 columns, 6000 nonzeros

Iteration    Objective       Primal Inf.    Dual Inf.      Time
       0    2.6436310e+33   1.963533e+33   2.643631e+03      0s
      29    6.1027866e+03   0.000000e+00   0.000000e+00      0s

Solved in 29 iterations and 0.00 seconds (0.00 work units)
Optimal objective  6.102786614e+03
Method: Dual Simplex
Optimal Profit: 6102.786614213885
Number of Selected Products: 5
Total Budget Used: 30000.0
Total Staff Hours Used: 512.709032502009
Total Display Space Used: 500.00000000000006
Total Advertising Used: 100.00000000000001
Sample Allocation (Top 10): {'x[84]': 39.46390443137951, 'x[299]': 24.264560357947214, 'x[330]': 52.03571878667337, 'x[763]': 5.994686477782236, 'x[854]': 51.52020040255822}
```